Go2 SDK Development Guide

Search article titles

About Go2
Remote Controller
SDK Concepts 3
Application Development 7
Software Interface Services 19
Motion Services Interface V2.0
DDS Services Interface
Avoidance Services Interface
VuiClient Interface
RobotStateClient Interface
Motion Switcher Service Interface
Sports Services Interface
AI motion Services interface
Basic Services Interface
LiDAR Services Interface
Multimedia Services Interface
UWB Services Interface
Fault Services Interface
SLAM and Navigation Services Interfa
Aruco Recharge Service Interface
Odometer Services Interface
D1 Mechanical Arm Services Interface
ROS2 Services Interface
Python Services Interface
Case reference 8
DDS Communication
High Motion Control
Basic Motion Control
Deployment Framework
Light Control
Get Remote Control Status
RL Control Routine
Exapmle of motor errors
Debugging Specification
FAQ

High Motion Control

Last Updated On : 2024-09-25 17:19:30

This article introduces how to use the Go2 SDK to call the high-level control interface of Go2.

High level control routine running test

The high-level routine files are located in `unitree_sdk2/example/go2/go2_sport_client.cpp`.

The beginning of the file lists test samples for various motion modes. The optional testing modes are divided into two parts: basic m ovements and special movements.

```cpp
enum test_mode
{
  /*---Basic motion control---*/
  normal_stand,      // standing normally
  balance_stand,     // balanced standing
  velocity_move,     // speed control
  trajectory_follow, // trajectory tracking control
  stand_down,        // Get down
  stand_up,          // Get down
  damp,              // damping mode, soft emergency stop
  recovery_stand,    // restore standing
  /*---Special motion ---*/
  sit,           // sit down
  rise_sit,      // recover from sitting state
  stretch,       // stretch
  wallow,        // roll
  content,       // Happy
  pose,          // pose
  scrape,        // Bowing to the New Year
  front_flip,    // front flip
  front_jump,    // Jump before 16
  front_pounce,  // pounce forward
  stop_move = 99 // Stop moving
};
```

Basic movements include:

Basic Sports Mode	Description
normal_stand	Normal standing, joint locking, standing high
balance_stand	Balanced standing, while maintaining balance in body posture
velocity_move	Move at the specified speed, at the given x, y, and aw speeds, with the reference coor dinate system being the body coordinate system
trajectory_follow	Track the given trajectory, track the given reference trajectory, and the reference coo rdinate system is the absolute coordinate system (with the starting position of the pr ogram as the origin)
stand_down	Lie down, joint locked
stand_up	Stand tall, joint locked
damp	Damping mode, all motors enter damping mode with the highest priority, equivalent t o a soft emergency stop
recovery_stand	Restore standing, restore standing from a overturned or prone state
stop_move	Stop all movements

Special actions include:

Special Action	Description
sit	Sit down
rise_sit	Restore from sitting state to previous state
stretch	Stretch
wallow	roll
content	Happy
pose	Pose
scrape	Bow to the New Year
front_flip	Front flip
front_jump	Jump forward
front_pounce	pounce forward

Modify variable TEST_MODE, you can select the example you want to run. For example, a demo for testing speed control:

```cpp
const int TEST_MODE = velocity_move;
```

After compiling the routine, use the remote control to put Go2 into motion mode, **ensure that the field is sufficiently open**, run th e program in the terminal, wait for 1 second, and Go2 will start executing the corresponding action. Press ctrl+c to terminate the pro gram.

This page contains content

High level control routine ...
Analysis of high-level cont...
Main program
Example of high-level control
High level status

Helpful
Feedback

```
~$ ./sportmode_test [network card name]
```

Analysis of high-level control routines

Main program

In the Custom class:

- The constructor Custom() calls sdk to complete the initialization of the sports client.
- RobotControl() calls the interface of SDK to implement various motion instances of Go2, and its code is explained in the high-level control instance.
- HighStateHandler() is a callback function for the high-level state, storing the obtained high-level state into the state variable.
- GetInitState() is used to obtain the initial state of the robot. Considering that the current position of the robot may not be near the origin of the odometer when running the program, for the sake of safety and intuition during the testing process, the testing routine will use the position and yaw angle of the program startup as the origin. Therefore, it is necessary to obtain the current position and yaw angle of the robot before the program starts.

```cpp
Custom()
{
  // initialization sportclient
  sport_client.SetTimeout(10.0f); // Request timeout
  sport_client.Init(); // initialization

  //Subscribe to high-level status topics to obtain robot status
  suber.reset(new unitree::robot::ChannelSubscriber<unitree_go::msg::dds_::SportModeState_>(TOPIC_HIGHSTATE));
  suber->InitChannel(std::bind(&Custom::HighStateHandler, this, std::placeholders::_1), 1);
};

// Callback function for high-level data reception
void HighStateHandler(const void *message)
{
  state = *(unitree_go::msg::dds_::SportModeState_ *)message;
  std::cout << "Position: " << state.position()[0] << ", " << state.position()[1] << ", " << state.position()[2
  std::cout << "IMU rpy: " << state.imu_state().rpy()[0] << ", " << state.imu_state().rpy()[1] << ", " << state
};

//Example of high-level control
void RobotControl(){...};

//Obtain initial state
void GetInitState()
{
  px0 = state.position()[0];
  py0 = state.position()[1];
  yaw0 = state.imu_state().rpy()[2];
  std::cout << "initial position: x0: " << px0 << ", y0: " << py0 << ", yaw0: " << yaw0 << std::endl;
};
```

In the main program, the robot will wait for 1 second to obtain a stable initial state before starting high-level control testing.

```cpp
int main(int argc, char **argv)
{
  if (argc < 2)
  {
    std::cout << "Usage: " << argv[0] << " networkInterface" << std::endl;
    exit(-1);
  }

  unitree::robot::ChannelFactory::Instance()->Init(0, argv[1]);
  Custom custom;

  sleep(1); // Wait for 1 second to obtain a stable state

  custom.GetInitState(); // Get initial position
  unitree::common::ThreadPtr threadPtr = unitree::common::CreateRecurrentThread(custom.dt * 1000000, std::bind(&

  while (1)
  {
    sleep(10);
  }
  return 0;
}
```

Example of high-level control

A branch statement was used in the test code to select the test instances that need to be run. The code in each case is explained as follows:

case Normal standing:

```cpp
case normal_stand:
  sport_client.SwitchGait(0); //idle gait
  sport_client.StandUp(); // Standing with locked joints
  break;
```

Case Balanced Standing:

- The range of Euler angles is: roll [-0.75, 0.75]; pitch[-0.75, 0.75]; yaw[-0.6, 0.6]
- The height of the fuselage is relative to the default state (0.33m), with a range of[-0.18, 0.03]

```cpp
case balance_stand:
  sport_client.Euler(0.1, 0.2, 0.3); // The input parameters are roll, pitch, and yaw angles, respectively
```

```
  sport_client.Euler(0.1, 0.2, 0.3); // The input parameters are roll, pitch, and yaw angles, respectively
  sport_client.BodyHeight(0.0);      // The relative height of the fuselage, 0 corresponds to 0.33m
  sport_client.BalanceStand();       // balanced standing
Break;
  break;
```

Case Speed Tracking:

```cpp
case velocity_move:
  sport_client.Move(0.3, 0, 0.3);     // The input parameters are vx, vy, and vyaw
  break;
```

case tracks the given trajectory:

Tracking a circular trajectory, the variable path contains 30 future expected path points as a reference for the trajectory tracking pro cess.

```cpp
case trajectory_follow: // 4. path mode walking
 time_seg = 0.2; //Time step of reference trajectory
 time_temp = ct - time_seg; //Current moment

 for (int i = 0; i < 30; i++)
 {
   time_temp += time_seg;

   //Calculate the path point of a circular trajectory based on the position of the program at runtime
   px_local = 0.5 * sin(0.5 * time_temp);
   py_local = 0.5 * cos(0.5 * time_temp)-1;
   yaw_local = 0;
   vx_local = 0.25 * cos(0.5 * time_temp);
   vy_local = -0.25 * sin(0.5 * time_temp);
   vyaw_local = 0;

   //Convert to Path Points in Absolute Coordinate System
   path_point_tmp.timeFromStart = i * time_seg;
   path_point_tmp.x = px_local * cos(yaw0) - py_local * sin(yaw0) + px0;
   path_point_tmp.y = px_local * sin(yaw0) + py_local * cos(yaw0) + py0;
   path_point_tmp.yaw = yaw_local + yaw0;
   path_point_tmp.vx = vx_local * cos(yaw0) - vy_local * sin(yaw0);
   path_point_tmp.vy = vx_local * sin(yaw0) + vy_local * cos(yaw0);
   path_point_tmp.vyaw = vyaw_local;
   path.push_back(path_point_tmp);
 }
 sport_client.TrajectoryFollow(path);
 break;
```

case Get down:

```cpp
case stand_down:
  sport_client.StandDown();
  break;
```

case Station height:

```cpp
case stand_up:
  sport_client.StandUp();
  break;
```

case damping mode, soft emergency stop:

```cpp
case damp:
  sport_client.Damp();
  break;
```

case Restore Standing:

```cpp
case recovery_stand:
  sport_client.RecoveryStand();
  break;
```

case Sit down:

Special actions can only be run once in a short period of time, so flag bits are used to determine whether the action is executed (the same below)

```cpp
case sit:
  if (flag == 0) // Running flag
  {
    sport_client.Sit();
    flag = 1;
  }
  break;
```

case Restore from sitting state to previous state:

```cpp
case rise_sit:
 if (flag == 0)
 {
   sport_client.RiseSit();
   flag = 1;
 }
 break;
```

case Stretch:

```cpp
case stretch:
 if (flag == 0)
 {
   sport_client.Stretch();
   flag = 1;
 }
 break;
```

case Roll:

```cpp
case wallow:
 if (flag == 0)
 {
   sport_client.Wallow();
   flag = 1;
 }
 break;
```

case Happy:

```cpp
case content:
 if (flag == 0)
 {
   sport_client.Content();
   flag = 1;
 }
 break;
```

case Pose:

```cpp
case pose:
 if (flag == 0)
 {
   sport_client.Pose(true);
   flag = 1;
 }
 break;
```

case Bowing to the New Year:

```cpp
case scrape:
  if (flag == 0)
  {
    sport_client.Scrape();
    flag = 1;
  }
  break;
```

case Front flip:

```cpp
case front_flip:
  if (flag == 0)
  {
    sport_client.FrontFlip();
    flag = 1;
  }
  break;
```

case Forward jump:

```cpp
case front_jump:
 if (flag == 0)
 {
   sport_client.FrontJump();
   flag = 1;
 }
 break;
```

case Forward thrust:

```cpp
case front_pounce:
 if (flag == 0)
 {
   sport_client.FrontPounce();
   flag = 1;
 }
 break;
```

case Stop movement:

```cpp
case stop_move: // stop move
  sport_client.StopMove();
  break;
```

High level status

The method to obtain the motion states of high-level buildings is as follows:

```cpp
TimeSpec stamp();
uint32_t errorCode(); // error code
IMU imuState(); // IMU status
uint8_t mode(); // Sports mode
/*
Sports mode
0. idle, default stand
1. balanceStand
2. pose
3. locomotion
4. reserve
5. lieDown
6. jointLock
7. damping
8. recoveryStand
9. reserve
10. sit
11. frontFlip
12. frontJump
13. frontPounc
*/

float progress(); // Action execution status: 0. dance false; 1. dance true
uint8_t gaitType(); // Gait type
/*
Gait type
0.idle
1.trot
2.run
3.climb stair
4.forwardDownStair
9.adjust
*/

float footRaiseHeight(); // Leg lift height
std::array<float, 3> position(); // 3D position
float bodyHeight(); // Body height
std::array<float, 3> velocity(); // Three-dimensional velocity
float yawSpeed(); // Yaw speed
std::array<float, 4> rangeObstacle(); // Obstacle distance
std::array<int16_t, 4> footForce(); // Four foot force
std::array<float, 12> footPositionBody(); // The position of the foot end relative to the body
std::array<float, 12> footSpeedBody(); // The speed of the foot relative to the body
std::array<PathPoint, 10>  pathPoints(); // The current tracked path point
```

For example:

```c
// Robot position
std::cout<<"position: "<<custom.state.position()[0]<<", "<<custom.state.position()[1]<<", "<<custom.state.posit

// Euler angle
std::cout<<"IMU rpy: "<<custom.state.imu_state().rpy()[0]<<", "<<custom.state.imu_state().rpy()[1]<<", "<<custor
```